Lab 4: Introduction to hardware description language (cont)

Assignments

Continuous assignment:

- Continuous assignments continuously monitor the RHS (right – hand side) and update the LHS (left – hand side) immediately when the RHS change, does not introduce any timing delays.
- Multiple continuous assignment executes concurrently, order of statements is not important.
- Used outside **always** statement to design **simple combinational circuit.**
- *Syntax: assign a = b;*

Blocking assignment:

- Cause the RHS expression to be evaluated immediately, and the LHS variable is updated before moving to the next statement.
- Multiple blocking assignments are executed sequentially in the order they appear in the code, order of statements is important.
- Used inside ***always*** statement to design **combinational circuit.**
- *Syntax: a = b;*

Non – blocking assignment:

- Updating for the variables on the LHS occur at the end of current time step, typically on a clock edge.
- Multiple non-blocking assignments executes concurrently, order of statements is not important.
- Used inside ***always*** statement to design more **complex sequential circuit.**
- *Syntax: a <= b;*

Compare between types of assignments

	Continous assignment	**Blocking assignment**	**Non-blocking assignment**
Update LHS to RHS	Immediately	Immediately	At the end of current time step, typically on a clock edge
Order of execution statements	Concurrently	In order	Concurrently
Using to design	Simple combinational circuit	Combinational circuit	Sequential circuit
Inside or outside ***always***	Outside	Inside	Inside
Syntax	assign a = b	a = b	a <= b

Always statement

Syntax:

always @(sensitivity list)
statement;

- Statements in *always* statement is executed only when the event specified in sensitivity list occurs.
- *Always* statement can be used to design flip-flop, latches or combinational circuit depend on the sensitivity list and statement.

- Types of always statement:

+ **always @(sensitivity list)** can be used to design sequential or combinational circuit based on sensitivity list

Ex: To design full adder: always@(a, b, cin)
To design D flip-flop: always@(posedge clk)

- **always_comb:** is dedicated for designing combinational circuit; reevaluates the statements inside *always_comb* any time the signal on RHS of statement changes.
- **always_latch** is dedicated for design latch
- **always_ff** is dedicated for design flip flop

Example

```
module assignment_example (
    input logic [3:0] a,        // 4-bit input
    input logic [3:0] b,        // 4-bit input
    input logic [3:0] c,        // 4-bit input
    output logic [4:0] sum_cont, // 5-bit output (Continuous assignment)
    output logic [4:0] sum1,     // 5-bit output (Blocking assignment)
    output logic [4:0] sum2);// 5-bit output (Blocking assignment)

// Continuous assignment
    assign sum_1 = a + b + c;

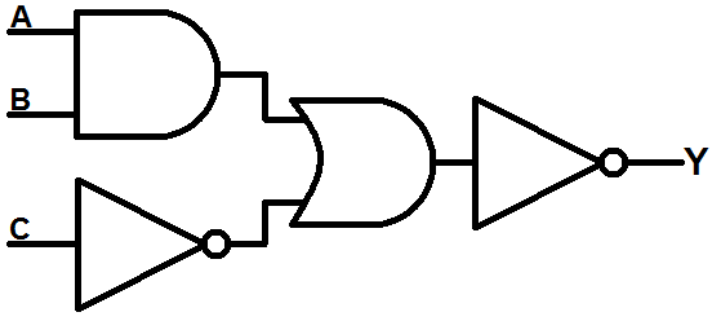
// Blocking assignments inside an always block
    always comb
    begin
        sum1 = a + b;         // First blocking assignment
        sum2 = sum1 + c;      // Second blocking assignment depends on sum1
    end
endmodule
```

Example

Example: design a simple combination circuit using always_comb statement

Using always_comb and blocking assignment
Using continuous assignment

```
module test(input logic a, b, c,
                  output logic y);
    logic p, g;
    always_comb
    begin
       p = a & b; // blocking
       g = p | (~c); // blocking
       y = ~g ; // blocking
   end
endmodule
```



If else statement

Syntax:

```
Always comb begin
        if (condition1) begin
        // Các câu lệnh được thực hiện khi condition1 là true
        end else if (condition2) begin
        // Các câu lệnh được thực hiện khi condition1 là false và
        condition2          là true
        end else if (condition3) begin
        // Các câu lệnh được thực hiện khi condition1 và condition2 đều
        false, và condition3 là true
        end else begin
        // Các câu lệnh được thực hiện khi tất cả các điều kiện trên
        đều false
        End
End
```

Case statement

```
Always comb begin
case (expression)
  value1: begin
    // Code block to execute when expression == value1
    //
   end
  value2: begin
    // Code block to execute when expression == value2
    //
    end
  // More value cases can be added here
  default:
    begin
    // Code block to execute when expression doesn't match any value
    // ...
    end
Endcase
end
```

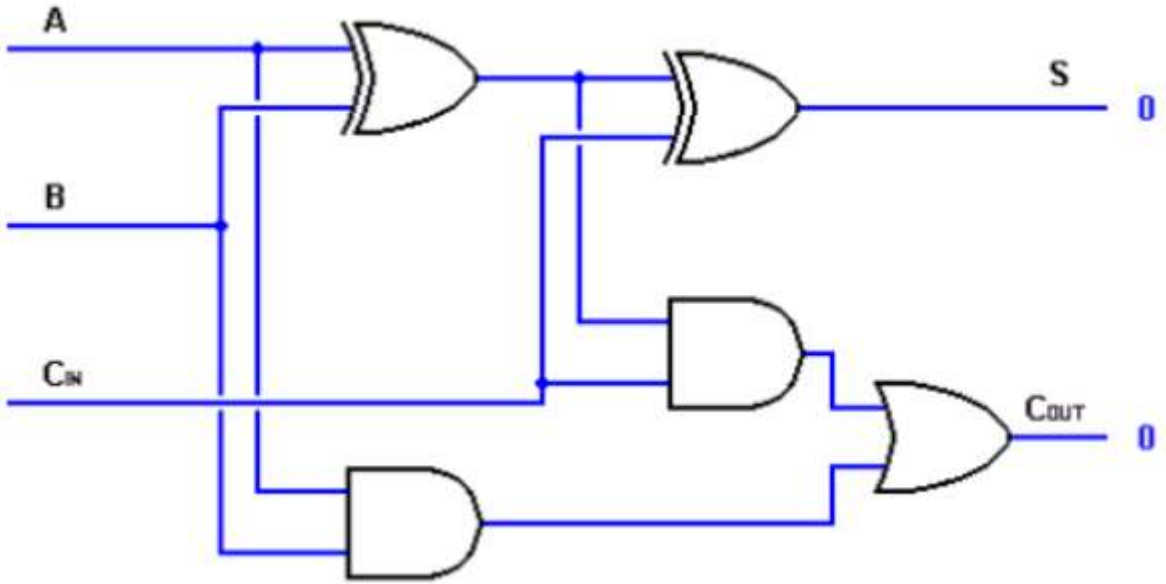
Ex1: Case statement and if else statement

Design a 2:4 decoder using SystemVerilog and write a Testbench to test it using case and if-else statement

- When the input is 00, the output will be 0001.
- When the input is 01, the output will be 0010.
- When the input is 10, the output will be 0100.
- When the input is 11, the output will be 1000.

Ex2 : Example on *fulladder*

Design a fulladder using always_comb statement



Exercise 3:

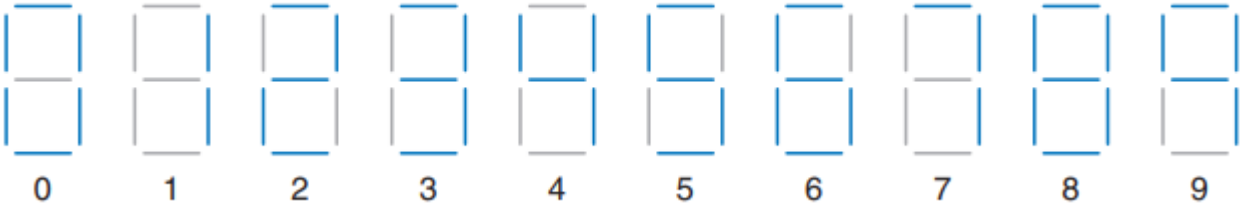
Design an ALU using in SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.

Hint: using *always_comb* statement, blocking assignment and case structure

Opcode	Operation
000	Add
001	Substract
010	And
011	Or
100	Xor
101	Equal

Exercise 4:

Design a seven segment decoder using always_comb statement and case structure in SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.
Hint: using always_comb statement and case structure



D_{4bit}	S_a	S_b	S_c	S_d	S_e	S_f	S_g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	0	0	1	1
others	0	0	0	0	0	0	0

